

文章编号:1006-2467(2015)12-1817-09+1832

DOI: 10.16183/j.cnki.jsjtu.2015.12.012

HIGHT 密码代数故障分析

陈浩¹, 王韬¹, 张帆^{2,3}, 赵新杰⁴, 孙也尊⁵

(1. 解放军军械工程学院 信息工程系, 石家庄 050003; 2. 浙江大学 信息与电子工程学系, 杭州 310027;
3. 保密通信重点实验室, 成都 610041; 4. 北方电子设备研究所, 北京 100083;
5. 北京军代局驻二四七军代室, 太原 030009)

摘要: 提出了一种轻量级分组密码 HIGHT 的代数故障分析方法, 并对其安全性进行了评估. 首先提出了一种高效的故障位置确定方法, 并分析了故障的最大注入深度; 然后利用代数方法构建 HIGHT 密码等效代数方程组, 通过故障攻击手段获取故障密文, 并对故障传播情况进行分析, 将涉及的全部密钥和加密中间状态用代数方程组表示, 实现故障信息最大化利用; 最后从理论上分析了攻击所需故障注入次数并在不同故障模型下进行了攻击实验, 使用 CryptoMinisat 解析器求解代数方程组恢复密钥信息. 结果表明: 与现有攻击相比, 该方法攻击所需故障注入次数少, 分析过程简单, 有效性和通用性好.

关键词: 轻量级分组密码; 代数分析; 故障分析; 代数故障分析

中图分类号: TP 309

文献标志码: A

Algebraic Fault Analysis of HIGHT

CHEN Hao¹, WANG Tao¹, ZHANG Fan^{2,3}, ZHAO Xin-jie⁴, SUN Ye-zun⁵

(1. Department of Information Engineering, Ordnance Engineering College, Shijiazhuang 050003, China;
2. Department of Information Science and Electrical Engineering, Zhejiang University, Hangzhou 310027, China;
3. Science and Technology on Communication Security Laboratory, Chengdu 610041, China;
4. Institute of North Electronic Equipment, Beijing 10083, China; 5. 247 Office, Beijing Military Representative Bureau of PLA, Taiyuan 030009, China)

Abstract: The security of HIGHT was evaluated using the algebraic fault analysis (AFA) for the first time. First, an efficient method to locate fault positions was proposed and the maximum fault injection round was analyzed. Secondly, HIGHT and keys that were affected during fault spreading were described as a set of algebraic equations. Finally, the number of fault injections was evaluated in theory and the attack was executed under several different fault models. Experimental results show the whole 128 bits keys can be recovered in 1.07 s when the fault depth is 6 rounds and the widths is 8 bits by injecting 15 random faults averagely, and the number is only 9 in the best case. The method can effectively and generally decrease fault injections, simplify the analysis process and can be applied into the algebraic fault attack of other block ciphers.

Key words: lightweight block cipher; algebraic analysis; fault analysis; algebraic fault analysis

收稿日期: 2015-01-05

基金项目: 国家自然科学基金项目 (61173191, 61272491, 61309021, 61472357, 61571063), 浙江大学中央高校基本科研专项资金项目 (2015QNA5005) 资助

作者简介: 陈浩 (1987-), 男, 湖北省武汉市人, 博士生, 主要研究方向为对称密码旁路分析.

张帆 (联系人), 男, 讲师, 电话 (Tel.): 0331-87994539; E-mail: fanzhang@zju.edu.cn.

故障攻击是一种利用密码实现所依附的物理设备平台在外界强电流、高电压、强辐射等手段干扰下可能出现的寄存器故障或运算错误来恢复密钥的攻击方法,最早由 Boneh 等^[1]于 1996 年提出,并成功用于基于中国剩余定理(Chinese Remainder Theorem, CRT)实现的 RSA 签名算法. 随后, Biham 等^[2]将故障攻击和差分分析的思想相结合,提出差分故障分析,并成功攻击了 DES(Data Encryption Standard)算法. 此后研究者将攻击对象扩展至 ECC^[3]等其他公钥密码, LED^[4]、PRESENT^[5]等分组密码, Grain^[6]等序列密码. 现有差分故障分析存在所需样本量大、故障传播情况需要手工分析、故障信息利用率低、通用性差等不足.

代数攻击是由 Courtois 在 2002 年提出的一种密码攻击方法,其主要思想是将一个密码算法的安全性完全归约到一个超定多变元高次方程系统的求解问题上,利用解析器进行方程的自动化求解,可充分利用计算资源. 然而,代数攻击中,由于方程个数和未知量个数较多,方程次数较高,使得在有限时间、空间、数据复杂度内实现方程组求解比较困难,极大地限制了代数攻击的有效性.

为了更好地发挥代数攻击和故障攻击的优势,在 eSmart 2010 会议上, Courtois 等首次将代数攻击和故障攻击结合,提出了代数故障分析方法,并成功用于 DES 算法,结果表明仅需穷举 24 位密钥并在加密 13 轮注入 1 bit 故障即可恢复全部密钥信息. 此后攻击对象趋于多样化,已扩展至 LED^[7]、PRESENT^[8]、Piccolo^[9]等分组密码, Trivium^[10]等序列密码. 同传统差分故障分析相比,代数故障攻击能够充分利用故障攻击中泄露出来的故障信息,具有攻击通用性强、方程可自动化求解的优点,为密码分析提供了一种新的自动化、通用化分析手段.

随着物联网逐步深入到人们生活的方方面面,如何在提高无线射频识别(RFID)标签、智能卡等资源受限设备执行效率的同时保障其安全性,成为亟待解决的问题. 在这种背景下,轻量级分组密码逐步成为密码学研究的新热点. 在 2006 年 CHES 会议上, HIGHT^[11]作为一种典型轻量级分组密码被提出. HIGHT 面向 8 位微处理器设计,非常适合在微型设备中使用,并且具有较好的硬件执行效率. 2009 年 Zhang 等^[12]应用饱和攻击分析方法,对前 22 轮 HIGHT 算法的安全性进行了分析,攻击需要选择 $2^{62.04}$ 个明文和 $2^{118.71}$ 次加密. 2012 年, 范伟杰等^[13]对 HIGHT 进行了故障攻击,攻击采用面向字节的故障模型,通过穷举 4 字节白化密钥,并分别在

倒数第 3 轮和倒数第 4 轮共注入约 32 个故障可恢复 HIGHT 全部 128 bit 密钥信息. 如何在现有研究基础上进一步简化故障分析过程,延伸故障注入深度,减小故障注入次数,增强攻击方法有效性是本文重点要解决的问题.

本文提出了一种针对 HIGHT 密码的代数故障分析方法,对其实现安全性进行了评估. 结果表明:故障注入深度为 6 轮,宽度为 8 bit 时,平均 15 次故障注入能在 1.07 s 内恢复全部 128 bit 主密钥信息,最好情况下仅需 9 次. 同现有 HIGHT 差分故障攻击^[13]相比,本方法无需穷举白化密钥即可恢复主密钥,所需故障注入次数较少,求解时间短,同时在多种不同故障模型下仍能成功实施攻击,有效性和通用性好.

1 HIGHT 算法

HIGHT 是一个 32 轮迭代轻量级分组密码,分组长度为 64 bit,密钥长度为 128 bit. HIGHT 采用广义 Feistel 结构,轮函数没有使用 S 盒,只采用循环移位、异或和模 2^8 加操作.

1.1 符号说明

田: 模 2^8 加运算.

⊕: 异或运算.

$A \lll S$: 循环左移 S 位.

$P = P_0 \parallel P_1 \parallel \cdots \parallel P_7$: 64 位加密明文.

$C = C_0 \parallel C_1 \parallel \cdots \parallel C_7$: 64 位加密密文.

$C' = C'_0 \parallel C'_1 \parallel \cdots \parallel C'_7$: 64 位错误密文.

$\Delta C_i = C_i \oplus C'_i$ ($0 \leq i \leq 7$): 密文差分值.

$X_i = X_{i,0} \parallel X_{i,1} \parallel \cdots \parallel X_{i,7}$ ($0 \leq i \leq 32$): 第 i 轮

8 Byte 中间输入.

$X'_i = X'_{i,0} \parallel X'_{i,1} \parallel \cdots \parallel X'_{i,7}$ ($0 \leq i \leq 32$): 注入故障后,错误的第 i 轮 8 Byte 中间输入.

MK_i ($0 \leq i \leq 15$): 16 Byte 主密钥.

δ_i ($0 \leq i \leq 127$): 轮常量.

SK_i ($0 \leq i \leq 127$): 128 Byte 子密钥.

WK_i ($0 \leq i \leq 7$): 8 Byte 白化子密钥.

1.2 HIGHT 加密过程

HIGHT 加密过程如图 1 所示,由初始变换、轮加密、未变换、密钥扩展 4 个子过程构成,具体执行过程如下:

步骤 1 明文 P 根据下式,经初始变换成加密首轮输入 X_0 .

$$\left. \begin{aligned} X_{0,4} &= P_4 \text{ 田 } WK_2; X_{0,5} = P_5 \\ X_{0,6} &= P_6 \oplus WK_3; X_{0,7} = P_7 \end{aligned} \right\} \quad (1)$$

步骤 2 算法执行 32 轮加密运算,将第 i

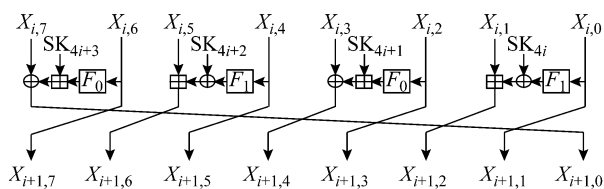


图 1 HIGHT 一轮加密过程

Fig. 1 Round function of HIGHT

($0 \leq i \leq 31$) 轮输出 X_i 按照式(2)转换成第 $i+1$ 轮输入 X_{i+1} . 每轮加密如图 2 所示, 使用 2 个线性函数 F_0 和 F_1 , 异或和模 2^8 加运算, 其中 F_0 和 F_1 表达式如式(3)所示.

$$\left. \begin{aligned} X_{i+1,0} &= X_{i,7} \oplus (F_0(X_{i,6}) \boxtimes SK_{4i+3}) \\ X_{i+1,1} &= X_{i,0} \\ X_{i+1,2} &= X_{i,1} \boxtimes (F_1(X_{i,0}) \oplus SK_{4i}) \\ X_{i+1,3} &= X_{i,2} \\ X_{i+1,4} &= X_{i,3} \oplus (F_0(X_{i,2}) \boxtimes SK_{4i+1}) \\ X_{i+1,5} &= X_{i,4} \\ X_{i+1,6} &= X_{i,5} \boxtimes (F_0(X_{i,4}) \oplus SK_{4i+2}) \\ X_{i+1,7} &= X_{i,6} \end{aligned} \right\} \quad (2)$$

$$\left. \begin{aligned} F_0(x) &= x^{<<<1} \oplus x^{<<<2} \oplus x^{<<<7} \\ F_1(x) &= x^{<<<3} \oplus x^{<<<4} \oplus x^{<<<6} \end{aligned} \right\} \quad (3)$$

步骤 3 算法在执行完 32 轮加密运算后, 将 X_{32} 根据下式, 经末变换转换成密文 C .

$$\left. \begin{aligned} C_0 &= X_{32,1} \boxtimes WK_4; C_1 = X_{32,2} \\ C_2 &= X_{32,3} \oplus WK_5; C_3 = X_{32,4} \\ C_4 &= X_{32,5} \boxtimes WK_6; C_5 = X_{32,6} \\ C_6 &= X_{32,7} \oplus WK_7; C_7 = X_{32,0} \end{aligned} \right\} \quad (4)$$

算法在初始变换和末变换中分别使用了 8 Byte 的白化密钥 WK_i ($0 \leq i \leq 7$), 每轮加密使用的 4 Byte 的子密钥 SK_i ($0 \leq i \leq 127$) 按照如下密钥扩展算法生成:

$$\left. \begin{aligned} WK_i &= MK_{i+12}, i = 0, 1, 2, 3 \\ WK_i &= MK_{i-4}, i = 4, 5, 6, 7 \end{aligned} \right\} \quad (5)$$

$$\left. \begin{aligned} SK_{16 \cdot i+j} &= MK_{j-i \bmod 8} \boxtimes \delta_{16 \cdot i+j} \\ SK_{16 \cdot i+j+8} &= MK_{(j-i \bmod 8)+8} \boxtimes \delta_{16 \cdot i+j+8} \end{aligned} \right\} \quad (6)$$

($0 \leq i, j \leq 7$)

根据式(6)可知, 恢复对应不同 16 Byte 主密钥的轮子密钥 SK_i 和白化密钥 WK_i 即可恢复全部主密钥信息.

2 HIGHT 代数故障分析

代数故障分析主要包括密码算法代数方程组构建、故障信息表示与利用、代数方程组求解 3 步. 攻

击的具体细节及步骤见文献[14]. 为了更加清楚直观地刻画 HIGHT 代数故障分析的相关问题, 给出以下定义.

定义 1 设密码算法 Δ 的加密轮数为 r , 攻击者在 t 时刻向 Δ 第 s ($1 \leq s \leq r$) 加密轮中间状态 X 注入随机故障 α , 生成错误密文 C' , 则称随机故障 α 作用 X 的比特数为故障宽度, 记为 f_w ; 称故障在传播过程中涉及的相关主密钥数为故障扩展度, 记为 f_s ; 称故障注入位置到错误密文输出之间的故障传播轮数为故障深度, 记为 $f_d = r - s$.

2.1 故障模型设定

(1) 对同一明文 P , 攻击者可以获得在同一个密钥 MK_i ($0 \leq i \leq 15$) 作用下的正确密文 C 和错误密文 C' .

(2) 在 HIGHT 加密第 r ($1 \leq r \leq 32$) 轮注入故障宽度 $f_w = 8$ 的随机故障, 故障值 φ 和故障位置 α 均未知, 故障深度 $f_d = 32 - r$.

2.2 故障位置确定

图 2 为 $(f_d, \alpha) = (6, 0), (6, 4)$ 时的故障传播路径图, 由图可知: 当故障深度 f_d 一定时, 注入位置 α 不同, 输出的错误密文 C'_i ($0 \leq i \leq 7$) 的个数和索引值 i 不同, 定义集合 $\Phi(f_d, \alpha) = \{i \mid \Delta C_i = 1, 0 \leq i \leq 7, 0 \leq f_d \leq 32, 0 \leq \alpha \leq 7 \text{ 且 } i, \alpha, f_d \in \mathbb{Z}\}$, 借助 $\Phi(f_d, \alpha)$ 可以准确定位故障注入位置 α , 集合 $\Phi(f_d, \alpha)$ 的计算可按照如下步骤进行:

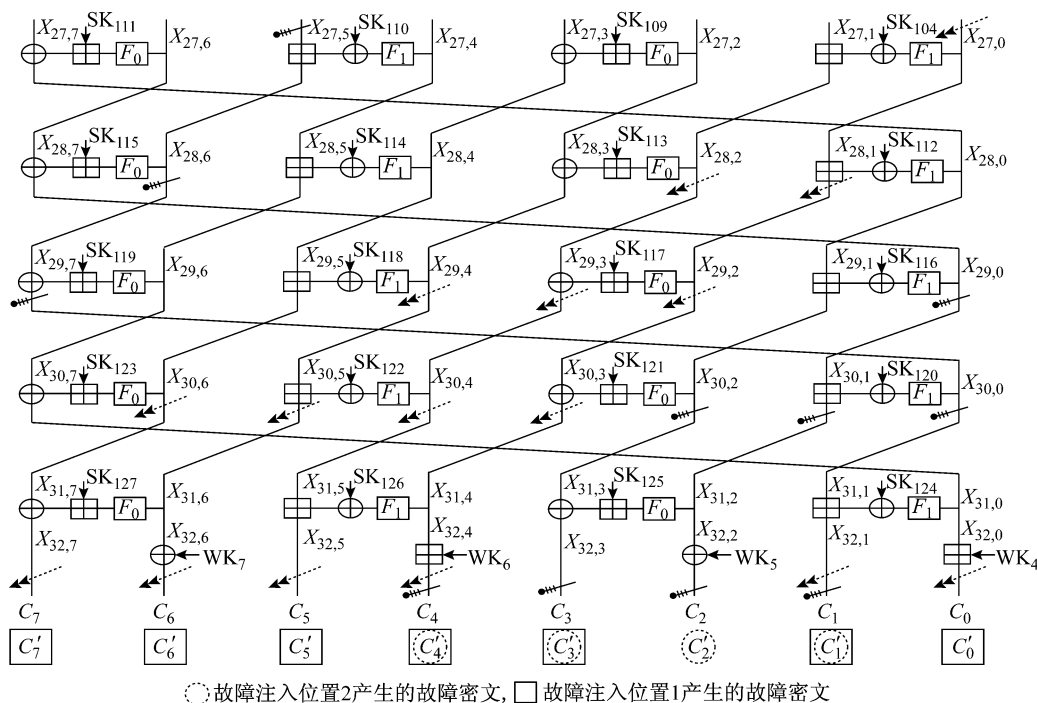
步骤 1 正确运行密码算法获取密文 C .

步骤 2 在 HIGHT 密码加密 $32 - f_d$ 轮第 α 个位置注入宽度为 8 bit 的故障, 获取故障密文 C' , 计算 $\Phi(f_d, \alpha)$.

步骤 3 改变故障注入位置 α , 并重复步骤 2, 直到 α 遍历 $X_{32-f_d} = X_{32-f_d,0} \parallel X_{32-f_d,1} \parallel \dots \parallel X_{32-f_d,7}$ 所有位置.

步骤 4 改变故障注入轮数 f_d , 并重复步骤 2 和 3, 直到 f_d 遍历所有加密轮, 输出 $\Phi(f_d, \alpha)$, 算法结束.

当 $f_d = 5, 6$ 时, 执行上述步骤输出结果如表 1 所示. 由表 1 可知, 当 $f_d \leq 6$ 且一定时, 对所有 $0 \leq \alpha_1 < \alpha_2 \leq 7$, $\Phi(f_d, \alpha_1) \cap \Phi(f_d, \alpha_2) = \emptyset$, 因此可根据 $\Phi(f_d, \alpha)$ 唯一确定故障注入位置 α . 当 $(f_d, \alpha) = (7, 0)$ 时, $\Phi(f_d, \alpha) = \Phi(7, 0) = \{0, 1, 2, \dots, 7\}$, 同理可得当 $\alpha = 2, 4, 6, 8$ 时, $\Phi(7, 0) = \Phi(7, 2) = \Phi(7, 4) = \Phi(7, 6)$, 此时 $\Phi(f_d, \alpha_1) \cap \Phi(f_d, \alpha_2) \neq \emptyset$, 无法据 $\Phi(f_d, \alpha)$ 唯一确定 α . 因此故障注入最大深度为 6, 当 $f_d \in [0, 6]$ 且一定时, α 可由 $\Phi(f_d, \alpha)$ 唯一确定.

图 2 $(f_d, \alpha) = (6, 0)$ 故障传播路线图Fig. 2 Fault propagation of $(f_d, \alpha) = (6, 0)$ 表 1 $\Phi(f_d, \alpha) (5 \leq f_d \leq 6, 0 \leq \alpha \leq 7)$ Tab. 1 $\Phi(f_d, \alpha) (5 \leq f_d \leq 6, 0 \leq \alpha \leq 7)$

α	$\Phi(f_d, \alpha)$	
	$f_d=6$	$f_d=5$
0	{0 1 4 5 6 7}	{3 4 5 6 7}
1	{0 1 5 6 7}	{4 5 6 7}
2	{0 1 2 3 6 7}	{0 1 5 6 7}
3	{0 1 2 3 7}	{0 1 6 7}
4	{0 1 2 3 4 5}	{0 1 2 3 7}
5	{1 2 3 4 5}	{0 1 2 3}
6	{2 3 4 5 6 7}	{1 2 3 4 5}
7	{3 4 5 6 7}	{2 3 4 5}

3 HIGHT 代数方程组构建

从本质上说,代数故障分析主要利用密码算法结构,密码正确/错误密文输出同故障传播中涉及的相关密钥之间的关系进行代数方程组构建,并通过解析器求解代数方程组进行密钥恢复.利用代数方法构建密码等效代数方程组,并将故障信息表示出来是代数故障分析很重要的一步.

3.1 密码代数方程组构建

为构建 HIGHT 整体代数方程组,共需引入 5 种类型 8 bit 中间变量.

for $(i=0; i < 31; i++)$

$X_{(i+1),0} = X_{i,7} \oplus (F_0(X_{i,6}) \boxtimes SK_{(4i+3)})$

$X_{(i+1),1} = X_{i,0}$

$X_{(i+1),2} = X_{i,1} \boxtimes (F_1(X_{i,0}) \boxtimes SK_{(4i)})$

$X_{(i+1),3} = X_{i,2}$

$X_{(i+1),4} = X_{i,3} \oplus (F_0(X_{i,2}) \boxtimes SK_{(4i+1)})$

$X_{(i+1),5} = X_{i,4}$

$X_{(i+1),6} = X_{i,5} \boxtimes (F_1(X_{i,4}) \boxtimes SK_{(4i+2)})$

$X_{(i+1),7} = X_{i,6}$

end

for $i=31$

$X_{(i+1),0} = X_{i,0}$,

$X_{(i+1),1} = X_{i,1} \boxtimes (F_1(X_{i,0}) \oplus SK_{124})$

$X_{(i+1),2} = X_{i,2}$,

$X_{(i+1),3} = X_{i,3} \oplus (F_0(X_{i,2}) \boxtimes SK_{125})$

$X_{(i+1),4} = X_{i,4}$,

$X_{(i+1),5} = X_{i,5} \boxtimes (F_0(X_{i,4}) \oplus SK_{126})$

$X_{(i+1),6} = X_{i,6}$,

$X_{(i+1),7} = X_{i,7} \oplus (F_0(X_{i,6}) \boxtimes SK_{127})$

end

式中: $i=0, 1, \dots, 31$ 表示加密轮数,每轮引入 8 个代数方程,40 个变量.

为构建未变换代数方程组,需引入 4 个白化密钥 $WK_j (4 \leq j \leq 7)$ 变量,共 32 bit.

$$\left. \begin{aligned} C &= C_7 \parallel C_6 \parallel C_5 \parallel C_4 \parallel C_3 \parallel C_2 \parallel C_1 \parallel C_0 \\ C_0 &= X_{32,0} \boxplus \text{WK}_4, \quad C_1 = X_{32,1} \\ C_2 &= X_{32,2} \oplus \text{WK}_5, \quad C_3 = X_{32,3} \\ C_4 &= X_{32,4} \boxplus \text{WK}_6, \quad C_5 = X_{32,5} \\ C_6 &= X_{32,6} \oplus \text{WK}_7, \quad C_7 = X_{32,7} \end{aligned} \right\} \quad (8)$$

3.2 模加方程的表示

模加运算是 HIGHT 算法中的唯一非线性部件,如何构建模加运算的代数方程组是 HIGHT 代数方程组构建的关键.定义 $X, Y, Z \in GF(2^n)$, $X = (x_{n-1}, x_{n-2}, \dots, x_0)$, $Y = (y_{n-1}, y_{n-2}, \dots, y_0)$, $Z = (z_{n-1}, z_{n-2}, \dots, z_0)$, x_0, y_0, z_0 分别表示 X, Y, Z 的最低位,则 $GF(2^n)$ 上的模 2^n 加运算可表示为:

$$\left. \begin{aligned} z_0 &= x_0 \oplus y_0 \\ z_1 &= x_1 \oplus y_1 \oplus c_1 \\ z_2 &= x_2 \oplus y_2 \oplus c_2 \\ &\vdots \\ z_i &= x_i \oplus y_i \oplus c_i \\ &\vdots \\ z_{n-1} &= x_{n-1} \oplus y_{n-1} \oplus c_{n-1} \end{aligned} \right\} \quad (9)$$

$$\left. \begin{aligned} c_0 &= 0 \\ c_1 &= x_0 y_0 \\ c_2 &= x_1 y_1 \oplus (x_1 \oplus y_1) c_0 \\ &\vdots \\ c_i &= x_{i-1} y_{i-1} \oplus (x_{i-1} \oplus y_{i-1}) c_{i-1} \\ &\vdots \\ c_{n-1} &= x_{n-2} y_{n-2} \oplus (x_{n-2} \oplus y_{n-2}) c_{n-2} \end{aligned} \right\} \quad (10)$$

式中, c_i 表示进位,且有 $c_0 = 0$,所以根据式(9)、(10)

可以将 $GF(2^n)$ 上的模 2^n 加运算转换成 $GF(2)$ 上的代数方程组.

3.3 密钥扩展算法

由 HIGHT 密钥扩展算法可知,轮子密钥的生成是由主密钥与已知常量 d_i 模加得到,因此在构建扩展密钥算法时仅需引入 2 种类型 8 bit 变量 SK_i 、 MK_i ,整个密钥扩展过程共引入 136 个变量.

$$\left. \begin{aligned} s_0 &= 0, s_1 = 1, s_2 = 0, s_3 = 1, s_4 = 1, s_5 = 0, s_6 = 1 \\ d_0 &= s_6 \mid s_5 \mid s_4 \mid s_3 \mid s_2 \mid s_1 \mid s_0 \\ \text{for}(i=1; i < 127; i++) \\ &S_{(i+6)} = S_{(i+2)} \oplus S_{(i-1)} \\ d_i &= s_{(i+6)} \parallel s_{(i+5)} \parallel s_{(i+4)} \parallel s_{(i+3)} \parallel s_{(i+2)} \\ &\parallel s_{(i+1)} \parallel s_0 \\ \text{end} \\ \text{for}(i=1; i < 7; i++) \\ &\text{for}(j=1; j < 7; j++) \\ &SK_{(16i+j)} = MK_{(j-i \bmod 8)} \boxplus d_{(16i+j)} \\ \text{end} \\ \text{for}(j=1; j < 7; j++) \\ &SK_{(16i+j+8)} = MK_{((j-i \bmod 8)+8)} \boxplus d_{(16i+j+8)} \\ \text{end} \\ \text{end} \end{aligned} \right\} \quad (11)$$

需要指出的是,在实际分组密码代数故障攻击中,一般仅需构建故障发生位置到密文输出之间的部分加密轮的代数方程组,无需构建 HIGHT 密码初始变化所对应的代数方程组,因此整个攻击过程实际构建的代数方程组个数为 $n=8(f_d+1)$.

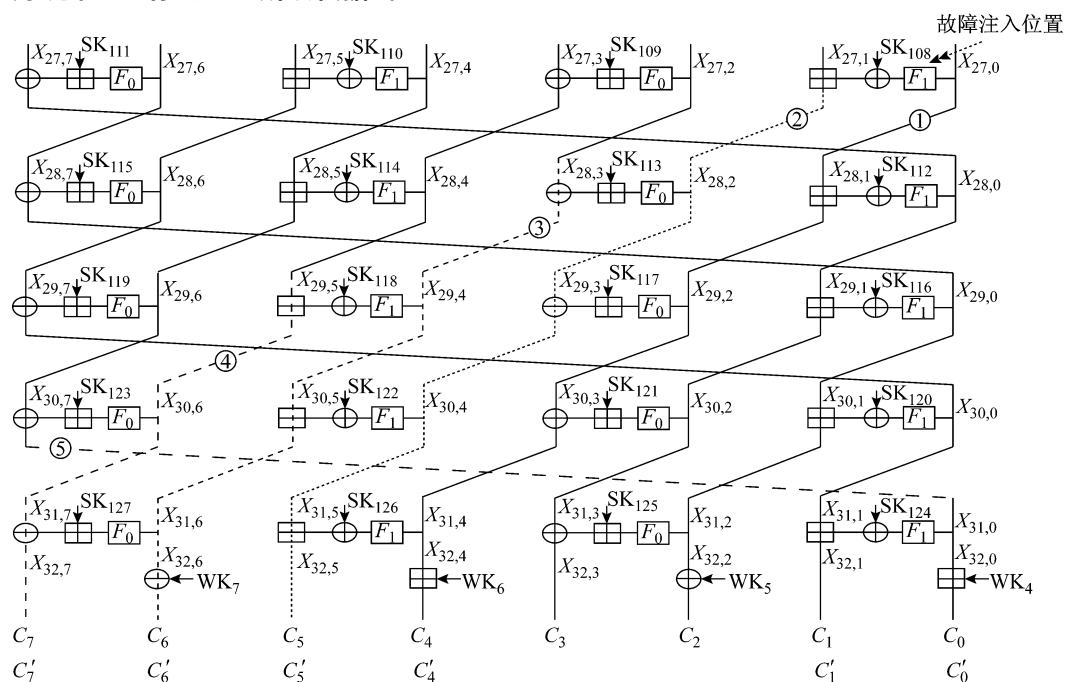


图 3 故障位置为 $X_{26,0}$ 时的故障传播路线

Fig. 3 Propagation of faults injected in $X_{26,0}$

3.4 故障信息利用与表示

根据 2.1 节的方法,当 $f_d \leq 6$ 时,攻击者可唯一确定故障位置 α ,因此对攻击者来说故障位置 α 已知.为了最大限度地利用故障信息,加速代数故障攻击的求解速度,需对故障注入之后的传播路径详细分析,以确定故障传播过程中涉及的相关密钥和中间变量信息,并用代数方程组表示出来.

可将故障传播过程细分为①~⑤共 5 条传播路径,如图 3 所示.将 5 条传播路径中全部受影响的相关密钥 SK_i 、 WK_i 以及中间状态值 X_i 表示出来,所构造的代数方程组为:

$$\left. \begin{aligned} X_{27,0} \oplus X'_{27,0} \oplus \varphi &= 0 \\ X'_{28,1} \oplus X'_{27,0} &= 0 \\ X'_{29,2} \oplus (X'_{28,1} \boxplus (F_1(X_{28,0}) \oplus SK_{112})) &= 0 \\ X'_{30,3} \oplus X'_{29,2} &= 0 \\ X'_{31,4} \oplus X'_{30,3} \oplus (F_0(X_{30,2}) \boxplus SK_{121}) &= 0 \\ X'_{32,4} \oplus X'_{31,4} &= 0 \\ C'_4 \oplus (X'_{32,4} \boxplus WK_6) &= 0 \end{aligned} \right\} \quad (12)$$

$$\left. \begin{aligned} X'_{28,2} \oplus (X'_{27,1} \boxplus (F_1(X'_{27,0}) \oplus SK_{108})) &= 0 \\ X'_{29,3} \oplus X'_{28,2} &= 0 \\ X'_{30,4} \oplus X'_{29,3} \oplus (F_0(X'_{29,2}) \boxplus SK_{117}) &= 0 \\ X'_{31,5} \oplus X'_{30,4} &= 0 \\ X'_{32,5} \oplus (X'_{31,5} \boxplus (F_1(X'_{31,4}) \oplus SK_{126})) &= 0 \\ X'_{32,5} \oplus C'_5 &= 0 \end{aligned} \right\} \quad (13)$$

$$\left. \begin{aligned} X'_{29,4} \oplus X'_{28,3} \oplus (F_0(X'_{28,2}) \boxplus SK_{113}) &= 0 \\ X'_{30,5} \oplus X'_{29,4} &= 0 \\ X'_{31,6} \oplus (X'_{30,5} \boxplus (F_1(X'_{30,4}) \oplus SK_{122})) &= 0 \\ X'_{32,6} \oplus X'_{31,6} &= 0 \\ X'_{32,6} \oplus WK_7 \oplus C'_6 &= 0 \end{aligned} \right\} \quad (14)$$

$$\left. \begin{aligned} X'_{30,6} \oplus (X'_{29,5} \boxplus (F_1(X'_{29,4}) \oplus SK_{118})) &= 0 \\ X'_{31,7} \oplus X'_{30,6} &= 0 \\ X'_{32,7} \oplus X'_{31,7} \oplus (F_0(X'_{31,6}) \boxplus SK_{127}) &= 0 \\ X'_{32,7} \oplus C'_7 &= 0 \end{aligned} \right\} \quad (15)$$

$$\left. \begin{aligned} X'_{31,0} \oplus X'_{30,7} \oplus (F_0(X'_{30,6}) \boxplus SK_{123}) &= 0 \\ X'_{32,0} \oplus X'_{31,0} &= 0 \\ (X'_{32,0} \boxplus WK_4) &= C'_0 \\ C'_1 \oplus (X'_{31,1} \boxplus (F_1(X'_{31,0}) \oplus SK_{124})) &= 0 \end{aligned} \right\} \quad (16)$$

由式(12)~(16)可知, $(f_d, \alpha) = (6, 0)$ 时所构造的代数方程十分简单,每注入一个故障引入 26 个新的变量共 128 bit. 同理,当故障位置 α 取其他值时,可构建对应的代数方程组.需要说明的是,当 f_d, f_w 和 α 为其他值时,上述方法依然适用,因此上述方法具有较好的通用性.

3.5 代数方程求解

本文选取 CryptoMinisat 解析器对构建的代数方程组进行求解.该解析器采用基于可满足性问题 SAT(Satisfiability)的代数方程组求解策略,具有高效、不受方程组次数约束的特点,在实际操作中需首先将代数方程组线性化,然后将线性化的代数方程组转化为符合 CryptoMinisat 输入格式的文本文件,以命令行方式调用 crypt. exe 进行自动求解.下面结合代数方程(17),进行简要说明.

$$x_0 \oplus x_1 \oplus x_0 x_1 \oplus x_0 x_1 x_2 = 0 \quad (17)$$

首先需要将式(17)转化成规定格式的文本文件,设 $q = x_0 x_1, s = x_0 x_1 x_2$ 则方程(17)可转化为:

$$\left. \begin{aligned} (x_0 \vee \bar{q}) \wedge (x_1 \vee \bar{q}) \wedge (\bar{x}_0 \vee \bar{x}_1 \vee q) \\ (x_0 \vee \bar{s}) \wedge (x_1 \vee \bar{s}) \wedge (x_2 \vee \bar{s}) \wedge \\ (\bar{x}_0 \vee \bar{x}_1 \vee \bar{x}_2 \vee \bar{s}) \end{aligned} \right\} \quad (18)$$

又可等效转化为:

$$\begin{aligned} x_0 \oplus x_1 \oplus x_0 x_1 \oplus x_0 x_1 x_2 &= \\ 0 \Leftrightarrow x_0 \oplus x_1 \oplus q \oplus s &= 0 \end{aligned} \quad (19)$$

根据 CryptoMinisat 的转化规则,需使用顺序的正整数对方程组中的常量 1 和所用变量进行编号,即:

$$\left. \begin{aligned} 1 \rightarrow 1, \quad x_0 \rightarrow 2, \quad x_1 \rightarrow 3 \\ x_2 \rightarrow 4, \quad q \rightarrow 5, \quad s \rightarrow 6 \end{aligned} \right\} \quad (20)$$

然后将式(18)中 7 个 CNF 范式和式(17)中的异或等式进行等效转换生成符合 CryptoMinisat 输入格式的文本文件,以命令行方式调用 crypt. exe 进行自动求解,求解结果为“SAT 1 2 3 4 5 6 0”,其中整数编号为正时表示与之对应的变量值为 1,否则为 0,所有编号 2~6 代表的变元 x_0, x_1, x_2, q, s 的求解结果都为 1.

4 复杂度分析

在代数故障分析中,通常使用故障注入次数 n 和密钥求解时间 t 来评估攻击的复杂度.故障扩散度 f_s 是刻画故障传播速度的一个重要指标,当 f_w 和 f_d 一定时, f_s 越大,故障在密码算法中的传播速度越快,涉及的相关密钥信息的比特数越多,恢复完整密钥信息所需的故障注入次数就越少;反之,则越多.本小节使用 f_s 从理论上对攻击所需的故障注入次数 n 进行评估.

定义 2 设 $U = \{i | 0 \leq i \leq 15, i \in \mathbb{Z}\}$, $\Omega(f_d, \alpha) = \{i | MK_i \text{ 为故障传播过程中涉及的主密钥}, 0 \leq i \leq 7, 0 \leq f_d \leq 32, 0 \leq \alpha \leq 7 \text{ 且 } i, \alpha, f_d \in \mathbb{Z}\}$, 则 $f_s(f_d, \alpha) = |\Omega(f_d, \alpha)|$, 且 $f_s(f_d, \alpha)$ 的加法定义为:

$$f_s(f_d, \alpha_1) + f_s(f_d, \alpha_2) = |\Omega(f_d, \alpha_1) \cup \Omega(f_d, \alpha_2)|$$

根据定义 2, 分析 $(f_d, \alpha) = (6, 0)$ 时的故障传播情况可知, $U - \Omega(6, 0) = \{4, 5, 9\}$, $f_s(6, 0) = 13$, 同理可得其他不同 (f_d, α) 下的 $U - \Omega(f_d, \alpha)$ (见表 2).

表 2 $U - \Omega(f_d, \alpha) (5 \leq f_d \leq 6, 0 \leq \alpha \leq 7)$

Tab. 2 $U - \Omega(f_d, \alpha) (5 \leq f_d \leq 6, 0 \leq \alpha \leq 7)$

α	$U - \Omega(f_d, \alpha)$	
	$f_d = 6$	$f_d = 5$
0	$\{4, 5, 9\}$	$\{0, 4, 7, 9, 12, 13\}$
1	$\{1, 4, 5, 9, 10\}$	$\{0, 4, 5, 7, 9, 12, 13, 14\}$
2	$\{4, 5, 6, 10\}$	$\{1, 4, 5, 9, 10, 14\}$
3	$\{2, 4, 5, 6, 7, 10, 11\}$	$\{1, 4, 5, 6, 9, 10, 14, 15\}$
4	$\{6, 7, 11\}$	$\{2, 4, 5, 6, 10, 11, 15\}$
5	$\{3, 6, 7, 11, 12\}$	$\{2, 4, 5, 6, 7, 8, 10, 11, 15\}$
6	$\{0, 7, 12, 13\}$	$\{3, 6, 7, 8, 11, 12\}$
7	$\{0, 4, 7, 9, 12, 13\}$	$\{0, 3, 6, 7, 8, 11, 12, 13\}$

由表 2 可得如下结论:

① 设 $\alpha \in \Gamma$, 当 $f_d = 6$, $\sum_{\alpha \in \Gamma} f_s(f_d, \alpha) = 16$ 时, $|\Gamma|_{\min} = 2$.

② 设 $\alpha \in \Gamma$, 当 $f_d = 5$, $\sum_{\alpha \in \Gamma} f_s(f_d, \alpha) = 16$ 时, $|\Gamma|_{\min} = 2$.

① 证明

$\sum_{\alpha \in \Gamma} f_s(f_d, \alpha) = 16 \Leftrightarrow \forall \alpha_1, \alpha_2 \in \Gamma$ 且 $\alpha_1 \neq \alpha_2$ 有 $(U - \Omega(f_d, \alpha_1)) \cap (U - \Omega(f_d, \alpha_2)) = \emptyset$ 成立

由表 2 知, $(U - \Omega(f_d, \alpha_1)) \cap (U - \Omega(f_d, \alpha_2)) = \emptyset$ 显然成立, 且 $|\Gamma|_{\min} = 2$ 的 Γ 集合有 7 个, $\Gamma_1 = \{0, 4\}$, $\Gamma_2 = \{0, 5\}$, $\Gamma_3 = \{0, 6\}$, $\Gamma_4 = \{1, 4\}$, $\Gamma_5 = \{1, 5\}$, $\Gamma_6 = \{1, 6\}$, $\Gamma_7 = \{2, 6\}$, 得证.

结论①表明, 当 $f_d = 6, \alpha \in \Gamma$ 时, 故障在传播过程中涉及 HIGHT 的全部主密钥信息, 通过求解代数方程组可以恢复全部主密钥信息. 下面对结论①成立时, 所需的故障注入次数 n 进行估计分析:

设故障注入次数为 $n (n \geq 2)$ 时, 结论①成立的概率为 P , 则当 $P \rightarrow 1$ 时, 可以认为在当前故障注入次数 n 下, 结论①成立. 由于很难使用严格的数学公式对 $P \rightarrow 1$ 时 n 的取值进行推导, 因此本文使用参数估计法对 n 的取值进行估计, 取置信水平为 95%, 结果如图 4 所示.

由图 4 可知, 当置信水平为 95% 时, $P \rightarrow 1$ 时 $n \rightarrow 11$, 由此可近似地认为, 当 $f_d = 6$, 随机注入 11 次宽度为 8 bit 的故障可恢复 HIGHT 密码全部密钥.

② 证明 由结论①的证明过程, 易证结论②成

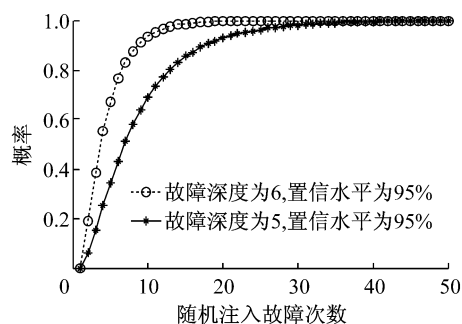


图 4 不同故障注入次数下恢复全部密钥概率

Fig. 4 Probability of recovery of entire keys at different fault injections

立, 且 $|\Gamma|$ 最小时的集合有 2 个, $\Gamma = \{2, 6\}, \{2, 7\}$. 置信水平为 95%, $P \rightarrow 1$ 时, $n \rightarrow 23$, 由此可以近似认为, 当 $f_d = 5$ 时, 需随机注入 23 次宽度为 8 的故障可以恢复全部密钥信息.

5 攻击实验及比较分析

在普通 PC 机上 (CPU 为 Intel(R) Core(TM) i5-3470 3.20 GHz, 内存 4 GB, Windows 7 32 bit 操作系统), 使用 C++ 编程语言模拟实现随机故障注入, 使用 CryptoMinisat 解析器 (版本号为 2.94) 进行密钥求解.

5.1 $f_w = 8, f_d = 6$ 故障模型下 HIGHT 攻击实例

(1) 首先随机产生一个明文 $P = 0F\ 14\ 16\ 05\ 0A\ 00\ 07\ 0C$, 主密钥为 $MK = 00\ 11\ 22\ 33\ 44\ 55\ 66\ 77\ 88\ 99\ AA\ BB\ CC\ DD\ EE\ FF$, 正确密文 $C = 00\ F4\ 18\ AE\ D9\ 4F\ 03\ F2$. 使用 3.2 节的方法构建正确加密时 HIGHT 密码完整代数方程组.

(2) 在 HIGHT 第 26 轮注入宽度为 8 的随机故障, 利用 2.2 节的方法确定故障注入位置, 并将从故障注入位置到错误密文输出之间的加密轮代数方程组表示出来, 在此基础上利用 3.4 节方法将故障信息表示出来.

(3) 使用 CryptoMinisat 解析器对所构建的代数方程组进行求解恢复主密钥信息, 求解结果如表 3 所示. 对主密钥 MK_i 进行编号, 编号时每字节主密钥信息占用 8 个编号位, 因此编号 2~129 依次表示 128 bit 主密钥信息, 求解的密钥为 $MK = 00000000\ 00010001\ 00100010\ 00110011\ 01000100\ 01010101\ 01100110\ 01110111\ 10001000\ 10011001\ 10101010\ 10111011\ 11001100\ 11011101\ 11101110\ 11111111$, 转化为 16 进制即 $00\ 11\ 22\ 33\ 44\ 55\ 66\ 77\ 88\ 99\ AA\ BB\ CC\ DD\ EE\ FF$, 同真实密钥信息一致, 攻击成功, 需要 13 个故障样本, 耗时 4.27 s.

表 3 实例中 CryptoMinisat 密钥求解结果

Tab. 3 Output of CryptoMinisat in one run of attack

编号	密钥/ bit	编号	密钥/ bit	编号	密钥/ bit	编号	密钥/ bit
-2	0	-34	0	-66	1	-98	1
-3	0	-35	1	-67	0	-99	1
-4	0	36	0	-68	0	100	0
-5	0	-37	0	69	0	101	0
-6	0	-38	0	-70	1	-102	1
-7	0	-39	1	-71	0	-103	1
-8	0	40	0	-72	0	104	0
-9	0	-41	0	73	0	105	0
10	0	42	0	74	1	106	1
-11	0	-43	1	-75	0	-107	1
-12	0	44	0	-76	0	108	0
-13	1	-45	1	77	1	109	1
14	0	46	0	78	1	110	1
-15	0	-47	1	-79	0	-111	1
-16	0	48	0	-80	0	112	0
-17	1	-49	1	81	1	113	1
-18	0	-50	0	-82	1	-114	1
19	0	51	1	83	0	115	1
-20	1	52	1	-84	1	116	1
-21	0	-53	0	85	0	117	0
-22	0	-54	0	-86	1	-118	1
23	0	55	1	87	0	119	1
-24	1	56	1	-88	1	120	1
-25	0	-57	0	89	0	121	0
26	0	58	0	90	1	122	1
27	0	59	1	91	0	123	1
-28	1	60	1	-92	1	124	1
-29	1	-61	1	93	1	125	1
30	0	62	0	94	1	126	1
31	0	63	1	95	0	127	1
-32	1	64	1	-96	1	128	1
-33	1	-65	1	97	1	129	1

攻击还产生 10 组不同的明文 P 和密钥 MK , 结果如图 5 所示, 由实验结果可知, 在实际攻击中平均 15 次故障注入即可恢复 HIGHT 全部密钥信息, 最好情况下仅需 9 次即可, 且故障注入次数 n 越大, 求解时间 t 越小。此外, 为了验证 3.4 节方法对故障信息利用的高效性, 在攻击实验中, 不同于 3.4 节构建故障传播过程中所有受影响密钥与中间状态的差分代数方程, 仅构建故障注入点的差分代数方程, 进行了 10 组实验, 结果如图 6 所示。

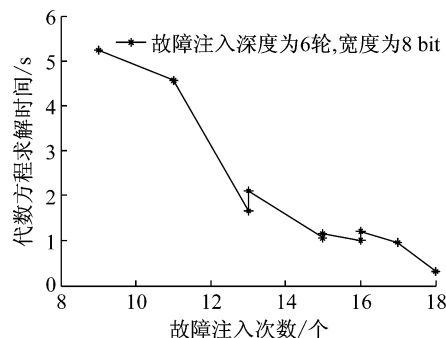


图 5 构建全部受影响密钥方程时的求解时间

Fig. 5 Solving time in the case of building the whole equations of keys effected

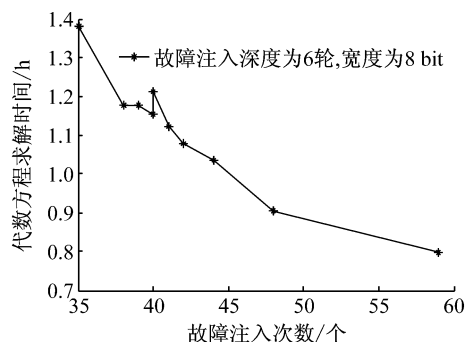


图 6 仅构建故障注入点方程时的求解时间

Fig. 6 Solving time in the case of only building equations of states with injected faults

对比图 5 与图 6 可见, 当仅构建故障注入点处的代数差分方程时, 恢复 128 bit HIGHT 主密钥, 需要的平均故障注入次数和代数方程组求解时间分别为 43 个和 3 957.8 s, 远大于使用 3.4 节方法时的 15 次和 1.93 s, 从而验证了 4.3 节方法利用故障信息的高效性。

5.2 不同故障模型下 HIGHT 攻击

为了验证本文方法在不同故障模型下的有效性, 在攻击实验中考虑了 4 种不同的故障模型, 每个故障模型下的实验次数 $N=100$, 结果如表 4 所示。

表 4 不同故障模型下攻击结果

Tab. 4 Attack results of different fault models

故障模型	故障实际 注入数/个	故障理论 注入数/个	平均密钥 求解时间/s
f_d f_w			
6 8	15	11	1.07
6 4	15	11	1.17
5 8	25	23	1.37
5 4	26	23	5.94

由表 4 可以得到如下结论:

(1) 当 f_d 相同时, f_w 越大, 故障影响到的相关密钥和中间状态值的比特位数越多, 恢复全部密钥信息所需的 n 就越小, 且 t 越小。

(2) 当 f_w 相同时, f_d 越大, 影响到的相关密钥和中间状态值的个数越多, 恢复全部密钥信息所需的 n 越小, 且 t 越小.

(3) 实际攻击中故障注入次数要大于理论分析值, 这是因为攻击实验中的故障注入是通过软件模拟实现的, 很难实现故障位置完全等概率分布, 因此, 在实际攻击中 n 应略大于理论注入次数, 表 4 的实验结果也证明了这一点.

5.3 与已有工作比较

与文献[13]中的故障攻击方法相比, 本文的方法具有以下优点:

(1) 有效性强, 攻击复杂度低. 文献[13]中的故障攻击方法能够成功恢复 HIGHT 密码全部 128 bit 密钥的前提是白化密钥 $WK_1 \sim WK_7$ 通过穷举方式已知, 故障的每个传播过程以及输入及输出差分值均已知. 由于 HIGHT 在设计时使用循环移位、异或和模 2^8 加的混合操作替代 S 盒, 因此故障传播特征复杂, 并且当故障注入轮数过大 ($f_d = 6$) 时, 故障的输入及输出差分值很难确定, 极大地限制了故障攻击的有效性. 本文的攻击方法能通过分析故障传播路径, 灵活地将故障传播过程中涉及的相关密钥和中间状态值用代数方程的形式表示, 最大限度地利用故障信息, 提高攻击效率, 减小故障注入次数(由 32 次减小为 15 次), 并且无需穷举白化密钥即可恢复全部 128 bit 密钥.

(2) 通用性更强. 相对于文献[13], 本文的攻击方法能够在 4 种不同故障模型实施, 成功恢复全部 128 bit 密钥信息, 并且给出的密码等效代数方程、模加方程及故障信息表示方法在不同故障模型下均适用, 因此具有更好的通用性.

6 结 语

本文对轻量级分组密码 HIGHT 首次提出一种代数故障攻击方法, 对 HIGHT 抗代数故障攻击能力进行了研究和实验. 在确定故障注入位置的基础上, 利用代数方法将密码和故障信息等效转化为代数方程组, 并从理论上分析了攻击所需故障注入次数, 最后在多种不同故障模型下进行了攻击实验. 结果表明: 在不同故障模型下本文攻击方法均能完全恢复密码的全部主密钥信息, 验证了方法的有效性和通用性, 说明 HIGHT 易受到代数故障攻击的威胁, 需要对密码设备加强保护以防止此类攻击.

下一步的研究工作包括采用效率更高, 求解时间更稳定的代数方程解析器, 如何进一步延伸故障注入轮数, 在故障位置未知情况下将故障信息用代

数方程组表示出来, 以及如何建立代数方程组、故障信息、密码算法三者之间的数学关系, 更加精确地评估代数故障攻击的复杂度.

参考文献:

- [1] Boneh D, DeMillo R A, Lipton R J. On the importance of checking cryptographic protocols for faults [C] // *Proceedings of the EUROCRYPT 1997*, LNCS 1233. Konstanz: Springer, 2001: 37-51.
- [2] Biham E, Shamir A. Differential fault analysis of secret key cryptosystems [C] // *Proceedings of the CRYPTO 1997*, LNCS 1294. Santa Barbara: Springer, 1997: 513-525.
- [3] Biehl I, Meyer B, Muller V. Differential fault attacks on elliptic curve cryptosystems [C] // *Proceedings of the CRYPTO 2000*, LNCS 1880. Santa Barbara: Springer, 2000: 131-146.
- [4] 李玮, 谷大武, 赵辰, 等. 物联网环境下 LED 轻量级密码算法的安全性分析 [J]. *计算机学报*, 2012, 35(3): 434-445.
LI Wei, GU Da-wu, ZHAO Chen, *et al.* Security analysis of the LED lightweight cipher in the Internet of Things [J]. *Chinese Journal of Computers*, 2012, 35(3): 434-445.
- [5] 赵光耀, 李瑞林, 孙兵, 等. Piccolo 算法的差分故障分析 [J]. *计算机学报*, 2012, 35(9): 1918-1926.
ZHAO Guang-yao, LI Rui-lin, SUN Bing, *et al.* Differential fault analysis on piccolo [J]. *Chinese Journal of Computers*, 2012, 35(9): 1918-1926.
- [6] Banik S, Maitra S, Sarkar S. A different fault attack on the grain family of stream ciphers [C] // *Proceedings of CHES 2010*, LNCS 7428. Santa Barbara: Springer, 2012: 122-139.
- [7] Zhang Fan, Zhao Xin-jie, Wang Tao, *et al.* Improved algebraic fault analysis: A case study on piccolo and applications to other lightweight block ciphers [C] // *Proceedings of COSADE 2013*, LNCS 7864. Paris: Springer, 2013: 62-79.
- [8] 吴克辉, 赵新杰, 王韬, 等. PRESENT 密码代数故障攻击 [J]. *通信学报*, 2012, 33(8): 85-92.
WU Ke-hui, ZHAO Xin-jie, WANG Tao, *et al.* Algebraic fault attack on PRESENT [J]. *Journal on Communications*, 2012, 33(8): 85-92.
- [9] 赵新杰, 郭世泽, 王韬, 等. Piccolo 密码代数故障分析研究 [J]. *计算机学报*, 2013, 36(4): 882-894.
ZHAO Xin-jie, GUO Shi-ze, WANG Tao, *et al.* Research of algebraic fault analysis on Piccolo [J]. *Chinese Journal of Computers*, 2013, 36(4): 882-894.

(下转第 1832 页)

参考文献:

- [1] Yu C B, Qin H F, Cui Y Z, *et al.* Finger-vein image recognition combining modified Hausdorff distance with minutiae feature matching[J]. **Interdisciplinary Sciences: Computational Life Sciences**, 2009, 1(4): 280-289.
- [2] Lin S, Yong-mei C, Nan L, *et al.* A scene matching method based on weighted Hausdorff distance combined with structure nformation[C]// **Control Conference (CCC), 2013 32nd Chinese**. Xi'an:IEEE, 2013: 5241-5244.
- [3] Wu J, Jing Z, Wu Z, *et al.* Study on an improved Hausdorff distance for multi-sensor image matching [J]. **Communications in Nonlinear Science and Numerical Simulation**, 2012, 17(2):513-520.
- [4] Ben-Avraham R, Henze M, Jaume R, *et al.* Minimum partial-matching and Hausdorff RMS-distance under translation: Combinatorics and algorithms [M]. Berlin: Springer Berlin Heidelberg, 2014: 100-111.
- [5] Nutanong S, Jacox E H, Samet H. An incremental Hausdorff distance calculation algorithm[J]. **Proceedings of the VLDB Endowment**, 2011, 4(8):506-517.
- [6] Zhu L, Zhu C. Application of Hausdorff distance in image matching[C]// **Electronics, Computer and Applications, 2014 IEEE Workshop on**. Ottawa:IEEE, 2014:97-100.
- [7] Zhang Y, Chen J, Wu J, *et al.* An Improved image retrieval method using curvature scale space[C]// **Proceedings of the 2012 Second International Conference on Electric Information and Control Engineering-Vol-ume 01**. Chongqing: IEEE Computer Society, 2012: 319-322.
- [8] Awrangjeb M, Lu G. Robust image corner detection based on the chord-to-point distance accumulation technique [J]. **Multimedia, IEEE Transactions on**, 2008, 10(6):1059-1072.
- [9] Yan Z, Gui-Li X, Yu-peng T, *et al.* Contour matching based on local curvature scale[C]// **Instrumentation, Measurement, Computer, Communication and Control (IMCCC), 2013 Third International Conference on**. Harbin:IEEE, 2013:1702-1707.
- [10] Maheshwari A, Sack J R, Shahbaz K, *et al.* Improved algorithms for partial curve matching[J]. **Algorithmica**, 2014, 69(3):641-657.
- [11] Huttenlocher D P, Klanderman G A, Rucklidge W J. Comparing images using the Hausdorff distance[J]. **Pattern Analysis and Machine Intelligence, IEEE Transactions on**, 1993, 15(9):850-863.
- [12] Sim D G, Kwon O K, Park R H. Object matching algorithms using robust Hausdorff distance measures [J]. **IEEE Transactions on Image Processing**, 1999, 8(3):425-429.
- [13] Zhu H, Zhang T, Yan L, *et al.* Robust and fast Hausdorff distance for image matching[J]. **Optical Engineering**, 2012, 51(1):1-5.

(上接第1825页)

- [10] Mohamed M S E, Bulygin S, Bchmann J. Using SAT solving to improve differential fault analysis of trivialum[J]. **International Journal of Security and Its Applications**, 2012, 6(1):29-38.
- [11] Hong D, Sung J, Hong S, *et al.* HIGHT: A new block cipher suitable for low-resource device[C]// **Proceedings of CHES 2006, LNCS 4249**. Yokohama: Springer, 2006:46-59.
- [12] Zhang P, Sun B. Saturation attack on block cipher HIGHT [C] // **Proceedings of CANS 2009, LNCS 5888**. Kanazawa:Springer, 2009:76-86.
- [13] 范伟杰, 吴文玲, 张蕾. HIGHT 算法的差分故障攻击[J]. **中国科学院研究生院学报**, 2012, 29(2):271-276.
FAN Wei-jie, WU Wen-ling, ZHANG Lei. Differential fault analysis on HIGHT[J]. **Journal of Graduate University of Chinese Academy of Sciences**, 2012, 29(2):271-276.
- [14] 郭世泽, 王韬, 赵新杰. 密码旁路分析原理与方法 [M]. 北京:科学出版社, 2014.